Homework #3: Migrating a Three-Tier Web Application to K3s

Course: Cloud Computing and Service Architectures (CCSA), Fall 2026
Topic: Container Orchestration with K3s

1. Objective

In HW#2, you developed a three-tier web application using Docker Compose. In this assignment, you will migrate the application to a **two-node K3s cluster** and automate its deployment.

The objectives are to:

- Understand Kubernetes Deployments, Services, and PersistentVolumeClaims (PVCs).
- Deploy and connect multiple application tiers in a Kubernetes cluster.
- Automate cluster setup, application deployment, and shutdown with simple commands.

Advanced scheduling, failover, and autoscaling will be explored in subsequent assignments.

2. Application Architecture

Reuse the HW#2 application, with PostgreSQL replacing SQLite3:

- **Web Tier:** Nginx — frontend/reverse proxy.
- **Application Tier:** Flask — REST APIs and application logic.
- **Database Tier:** PostgreSQL — persistent data storage.

Use maintained, lightweight container images. You may choose suitable base images and versions; no specific Linux distribution is required.

Target Environment:

- One K3s control-plane node and one worker node.
- Three Kubernetes Deployments, each with one replica.
- Kubernetes Services for inter-tier communication.
- Persistent storage for PostgreSQL.

3. Assignment Requirements

Part A. K3s Cluster Setup and Automation

Set up a two-node K3s cluster using either approach:

Option 1: Vagrant-based VMs

- Provide a `Vagrantfile` to create one control-plane VM and one worker VM.
- `vagrant up` must automatically provision K3s, join the worker, and deploy the three-tier application.
- `vagrant halt` must shut down both VMs; a subsequent vagrant up must restore the working environment.
- `vagrant destroy -f` should remove the VMs when the environment is no longer needed.

Option 2: Existing Linux Machines/VMs

- Use two existing Linux hosts (physical machines or VMs).
- Provide `k3s-up.sh` and `k3s-down.sh` scripts.
- `./k3s-up.sh` must configure/start the K3s cluster and deploy the application.
- `./k3s-down.sh` must stop the application and K3s services without deleting persistent data.

Both options must support repeatable startup and shutdown without requiring manual cluster reconfiguration. Document any prerequisites and verify that both nodes reach the Ready state.

Part B. Application Migration

1. **Container Images**
    - Reuse Nginx and Flask application components from HW#2.
    - Replace `SQLite3` with `PostgreSQL` and update the Flask database connection.
    - Make all images accessible to the K3s cluster.
2. **Kubernetes Resources**
    - Create three Deployments: web, api, and db, with one replica each.
    - Create a `NodePort` Service for Nginx.
    - Create `ClusterIP` Services for Flask and PostgreSQL.
    - Use Kubernetes Service names for communication between tiers.
3. **Persistent Storage**
    - Create a PVC using K3s's default local-path StorageClass.

    - Mount the PVC into PostgreSQL's data directory.
    - Preserve database records across PostgreSQL Pod recreation.
4. **Deployment Automation**
    - Include application deployment in the automated startup procedure.
    - Use YAML manifests and `kubectl`, rather than Docker Compose, to deploy the application.

Part C. Verification

Demonstrate:

1. **Cluster Startup:** One command brings up the cluster and application; both nodes are Ready.
2. **Application Functionality:** Access the web application through the Nginx `NodePort` and execute its functions.
3. **Inter-Tier Connectivity:** Nginx communicates with Flask, and Flask accesses PostgreSQL using Kubernetes Services.
4. **Data Persistence:** Insert a record, delete the PostgreSQL Pod, and verify that the record persists after the Pod is recreated.
5. **Cluster Restart:** Shut down and restart the environment using the required commands; verify that the application and database remain functional.

4. Deliverables

Submit one ZIP file named `HW3_StudentID.zip` containing:

Item	Required Content
README.md	Prerequisites, one-command startup/shutdown, deployment instructions, application URL
Vagrantfile or scripts/	Vagrant configuration and provisioning scripts, or k3s-up.sh / k3s-down.sh
architecture.pdf	One-page diagram showing nodes, three tiers, Services, and persistent storage
k8s/	All Kubernetes YAML manifests
src/	Updated application code and Dockerfiles

evidence/	Command outputs/screenshots demonstrating all five verification requirements
comparison.md	≤300-word comparison of Docker Compose and K3s

Reproducibility: The instructor must be able to start, test, stop, and restart the environment using the documented commands.

5. Grading Criteria

Component	Weight
K3s cluster setup and lifecycle automation	30%
Application migration and Kubernetes resources	30%
Service networking and database persistence	25%
Verification, documentation, and reproducibility	15%
Total	**100%**

6. Scope and Constraints

- Use a two-node K3s cluster and a three-tier application.
- Use PostgreSQL, with one `Deployment` replica per tier.
- Do not use Docker Compose for the K3s application deployment.
- Preserve database data across Pod recreation and normal environment shutdown/restart.
- No requirement for automatic failover, cross-node storage migration, autoscaling, or high availability.
- Advanced Kubernetes orchestration will be addressed in subsequent assignments.

Expected Learning Outcome: Students should be able to migrate a multi-container application from Docker Compose to K3s and automate its complete deployment lifecycle using reproducible commands.